

GUIA TÉCNICO DO PORTFOLIO

Como o site funciona, onde editar, como publicar, testar, fazer backup e entregar o projeto para uma pessoa desenvolvedora ou outra IA sem perder contexto.

ESTADO AUDITADO	14 SET 2026
3 seções · 10 projetos publicados · 18 migrations	33 testes aprovados · 9 específicos de viewport pulados

Resumo: o código atende ao propósito do portfolio. Ele é um site estático em HTML/CSS/JavaScript, usa Supabase como CMS e autenticação, Vercel para deploy, Cloudflare para DNS/mídia R2, Resend para e-mail e Playwright para testes.

RAIZ DO PROJETO

/Users/vfx4/Documents/GitHub/portfolio

O sistema em uma página

Fluxo público

Navegador → arquivos estáticos da Vercel → leitura pública do Supabase → montagem da interface no navegador.

Fluxo editorial

Admin autenticado → alterações no Supabase → conteúdo público muda imediatamente. O botão “Update SEO & Previews” aciona um novo deploy para atualizar HTML indexável, sitemap e previews sociais.

Camada	Responsabilidade	Arquivos/serviço
Interface	Layout, tipografia, animações, responsividade	*.html, styles.css, components/
Orquestração	Descobre a rota, carrega dados e monta páginas	script.js, portfolio.js, project-detail.js
Conteúdo	Projetos, seções, textos globais e traduções	Supabase + data/ + config/
Admin	Login, edição, upload, ordenação, backups e deploy SEO	admin.html, admin/
Build	Gera dist/, sitemap, metadados e snapshot público	scripts/build-static.mjs
Infra	Deploy, domínio, mídia e envio de contato	Vercel, Cloudflare/R2, Supabase, Resend

Fonte de verdade e fallback

Supabase é a fonte viva. Em cada build, os registros publicados também viram um snapshot em `dist/data/projects-data.js` e `dist/config/page-configs.js`. Esse snapshot pinta conteúdo correto imediatamente e sustenta a página se a primeira consulta remota falhar. No código-fonte, `PROJECTS_DATA` fica vazio para que projetos-template nunca reapareçam.

Na Vercel, se o snapshot não puder ser atualizado, o build novo falha e o último deploy válido continua online.

Onde cada coisa mora

Caminho	Use para
index.html	Tela inicial, ENTER, WATCH REEL e seletor BR/EN.
gallery.html	Overview /work e páginas de seleção de projetos.
project.html	Estrutura da página individual de projeto.
contact.html	Formulário público de contato.
styles.css	Todo o visual público e breakpoints mobile.
portfolio.js	Troca de seção sem recarregar o menu; busca/fallback de conteúdo.
project-detail.js	Mídia, texto, stills, modal do YouTube e metadata do projeto.
components/	Peças isoladas: cards, hero, rotas, i18n, preloads e formulários.
config/	Defaults do site, Supabase público e seções-fallback.
data/	Cliente Supabase e fallback de projetos.
admin/	Interface editorial, estado local e export/import de backup.
supabase/migrations/	Evolução versionada do banco: execute em ordem numérica.
supabase/functions/	E-mail de contato e gatilho protegido de deploy.
api/	Health check chamado pelo cron da Vercel.
tests/	Testes Playwright e testes unitários do health check.
dist/	Saída gerada. Não edite manualmente.

Regra prática

- Conteúdo: altere pelo admin.
- Estética/comportamento: altere HTML/CSS/JS do repositório.
- Banco: adicione uma nova migration; não reescreva migrations já aplicadas.
- Produção: publique por commit/push; nunca edite dist/ como fonte.

Do clique à página pronta

1. Homepage

site-settings.js aplica tema/textos e carrega configurações públicas. O preview do reel fica separado do vídeo completo; WATCH REEL carrega a versão adequada ao viewport somente quando necessário.

2. Entrada em /work

section-entry-preview.js antecipa os destaques. portfolio.js monta o Featured Hero, o texto editorial e a lista de seções. O menu permanece montado ao trocar filtros.

3. Seleção de projetos

project-gallery.js filtra, ordena e distribui cards em masonry no desktop. No mobile volta a um fluxo vertical. project-frame.js controla imagem/vídeo de hover e salva um preview do projeto clicado.

4. Projeto individual

project-preview.js pinta título, categoria e cover a partir de sessionStorage antes da rede. project-detail.js substitui esse preview pelos dados finais do Supabase e monta vídeo, stills, créditos e descrição.

5. Rotas limpas

components/routes.js converte IDs/slugs em /work/slug e /project/slug. vercel.json cuida de rewrites, redirects antigos e headers de segurança.

URL	Tela
/	Homepage
/work	Overview editorial + Featured Hero
/work/{slug-da-seção}	Galeria filtrada
/project/{slug-do-projeto}	Projeto individual
/contact	Formulário
/admin	Painel protegido

Workflow editorial recomendado

Editar e salvar

- Entre em /admin com o usuário cadastrado em Supabase Auth e listado em portfolio_admins.
- Escolha a seção, selecione/arraste um projeto e edite os campos. PT-BR é o conteúdo principal; EN é opcional.
- Salve. O admin grava primeiro a cópia local e depois sincroniza com Supabase; leia a mensagem de status.
- Uploads vão ao bucket portfolio-media. Vídeos grandes podem usar URL pública do Cloudflare R2.
- Use “Update SEO & Previews” depois de mudar título de aba, slug, descrição ou imagem social.

Backups

- Export Full Backup: JSON restaurável pelo próprio admin, com inventário de URLs de mídia.
- Export SQL Backup: conteúdo textual/estrutural restaurável pelo SQL Editor após aplicar migrations.
- `npm run backup:media -- /caminho/backup.json` baixa cópias locais das mídias referenciadas.

O que não está no backup

Senhas, usuários Auth, secrets, mensagens de contato e configuração externa de Vercel/Cloudflare/Resend. Guarde essas credenciais em um gerenciador de senhas e documente a propriedade das contas.

Conteúdo para revisar

Na auditoria de 14/09/2026, “QUANTUM FLUIDS” ainda estava publicado no Supabase em Digital Alchemy e usava o vídeo local bg-cinema.mp4. Confirme no admin se é projeto real ou placeholder; ele não foi removido automaticamente.

Rodar, testar e publicar

Pré-requisitos

Node.js 18 ou superior, Git e acesso à pasta do repositório. A chave presente em config/supabase-config.js é publishable/anon e pode existir no navegador; nunca coloque service_role ali.

Instalação e validação

```
cd /Users/vfx4/Documents/GitHub/portfolio
npm ci
npm test
```

npm test executa o build, 3 testes unitários do health check e a suíte Playwright em desktop e mobile. O estado auditado passou 33 testes; 9 foram pulados intencionalmente por pertencerem ao outro viewport.

Build manual

```
npm run build
```

A saída fica em dist/. O build busca os registros publicados, gera rotas HTML, canonical/OG metadata, sitemap e o snapshot de runtime. Para testar apenas o navegador: npm run test:e2e.

Publicação

Commit → push para main no GitHub → GitHub Actions testa → Vercel faz build/deploy → arturaraujo.com serve o novo deploy. Alterações comuns do admin ficam visíveis via Supabase sem commit; SEO/previews exigem o deploy hook.

Antes de entregar

- git status deve conter somente mudanças intencionais.
- npm audit deve informar zero vulnerabilidades ou ter cada exceção documentada.
- Teste homepage, menu, três seções, pelo menos um projeto com e sem YouTube, contato e admin.
- Teste 320 px, 390 px e desktop, inclusive orientação/reel no celular real.

O que pode e não pode ir ao Git

Pode ser público	Deve ser secreto
SUPABASE_URL	Supabase service_role key
Supabase publishable/anon key	RESEND_API_KEY
URLs públicas de imagens/vídeos	CONTACT_TO_EMAIL
Slugs, títulos e descrições	CONTACT_RATE_LIMIT_SECRET
Domínio e sitemap	VERCEL_DEPLOY_HOOK_URL e CRON_SECRET

Proteções existentes

- RLS: público lê apenas conteúdo publicado; escrita exige usuário admin.
- CSP, HSTS, X-Frame-Options, nosniff, Referrer-Policy e Permissions-Policy em vercel.json.
- Contato com validação, limite de tamanho, honeypot, escaping, rate limit e secrets server-side.
- Deploy hook exige sessão válida e membership em portfolio_admins.
- Health endpoint exige CRON_SECRET e faz somente três leituras públicas pequenas.
- Dependência @playwright/test fixada; Supabase SDK fixado em 2.112.4 no CDN.

Risco operacional restante

CONTACT_ALLOWED_ORIGINS e ADMIN_ALLOWED_ORIGINS precisam continuar configurados nos secrets. A ausência dessas listas reduz a proteção de origem. Origin não substitui autenticação, mas deve ser mantido como defesa adicional.

Resultado da auditoria

npm audit: 0 vulnerabilidades. Não foram encontrados service-role keys, tokens Resend ou deploy hooks no código público. Os usos de HTML dinâmico auditados escapam o conteúdo ou usam textContent.

Se Supabase ou a IA desaparecer

Se Supabase ficar temporariamente indisponível

O deploy publicado mantém o snapshot dos projetos/seções e páginas HTML geradas. O conteúdo pode continuar visível, mas edição, autenticação, formulário e atualização em tempo real dependem do serviço. Não faça um novo deploy até o Supabase voltar; na Vercel, o build agora recusa snapshot vazio.

Se for migrar de backend

- Exporte JSON + SQL no admin e baixe as mídias.
- Recrie as entidades sections, projects, site_settings, admins e contact_messages.
- Implemente a mesma interface esperada por data/supabase-service.js ou substitua esse adaptador.
- Preserve IDs internos e slugs; previousSlugs mantém redirects antigos.

Se outra pessoa ou IA assumir

Abra a mesma pasta Git, leia este PDF, DEPLOYMENT.md e supabase/README.md. Depois examine package.json, vercel.json, scripts/build-static.mjs, portfolio.js, data/supabase-service.js e admin/admin.js. Rode npm ci e npm test antes de editar. Preserve mudanças locais, nunca exponha secrets e não altere migrations já aplicadas.

Prompt de handoff sugerido

Trabalhe no repositório /Users/vfx4/Documents/GitHub/portfolio. É um portfolio estático com Supabase CMS, Vercel, Cloudflare/R2, Resend e Playwright. Leia primeiro o guia PDF, DEPLOYMENT.md e supabase/README.md. Preserve o ID visual editorial, o menu persistente, as rotas limpas e os fallbacks de preload. Antes de mudar, verifique git status; depois rode npm test e relate arquivos alterados, riscos e passos externos necessários.

Manutenção sem surpresas

Toda alteração de código

- Leia a implementação existente antes de adicionar outra camada.
- Prefira componentes e helpers atuais; evite duplicar fetch, rotas, normalização e escaping.
- Mantenha desktop e mobile na mesma alteração e inclua teste quando houver regressão possível.
- Rode `git diff --check`, `npm test` e revise o site real após o deploy.

Todo mês ou após mudanças grandes

- Exporte backup JSON e SQL; guarde fora do computador do projeto.
- Baixe mídias importantes e confirme que URLs R2/Supabase continuam públicas.
- Confira GitHub Actions, Vercel Deployments/Cron, Supabase logs e Resend delivery.
- Rode `npm audit` e atualize dependências com teste completo.

Ao adicionar uma feature

- Se houver novo campo de conteúdo: migration → mapeamento row/camelCase → admin → render público → backup SQL/JSON → build snapshot → teste.
- Se houver nova página: HTML → rota/rewrites → canonical/OG → sitemap → navegação → mobile → teste 404/back.
- Se houver novo serviço externo: documento conta, variáveis, limites, recuperação e custo.

Conclusão

O projeto está adequado para o portfolio proposto e tem boa separação entre conteúdo, interface e infraestrutura. A manutenção é simples desde que Supabase/migrations, build estático e testes continuem sendo tratados como um único fluxo.